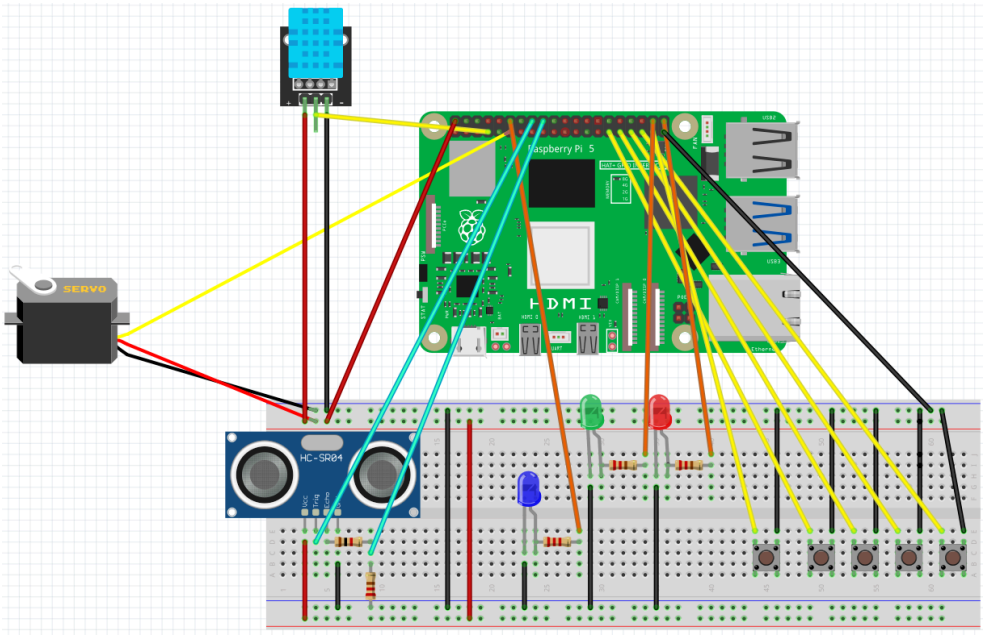
PROJECT SPECIFICATIONS

Smart Home Platform for Sensor Data Logging and Device Control	
PROJECT OBJECTIVE	*Smart home system based on Raspberry Pi that reads data from sensors and controls basic devices. The user will be able to check temperature, humidity, motion, and door status, as well as open the door remotely. It includes a custom 5-button keypad for entering door codes, with LED feedback and a servo-controlled door lock. Code-based door opening automatically synchronizes the state with the web dashboard. Selected sensor data will be stored in a database for the web application.*
OVERVIEW DIAGRAM (SIMPLIFIED)	
 **Figure 1. Hardware schematic of the Raspberry Pi-based smart-home system.**	
HARDWARE PLATFORM USED, MEASUREMENT COMPONENTS AND EXECUTIVE	*Raspberry Pi 5, HC-SR04 ultrasonic distance sensor, DHT11 sensor module (temp. and humidity measurements), servomechanism SG90, LED indicators, push buttons, resistors, jumper wires and breadboard*

1. Purpose and scope of the project. **2**

2. Scheme* **4**
3. Design assumptions vs. implementation. **4**
4. Code listing. **5**
4.1. Main Server (Spring Boot) 5
4.2. Code for Raspberry PI 5 7
5. Photos of the completed layout. **9**
6. Screenshots of the application. **10**
7. Summary and conclusions. **11**

1. Purpose and scope of the project.

Developed project involved several modules that we will call consistently as follows:

Device

Raspberry Pi 5 equipped with sensors and actuators connected to the following GPIO pins:

- **DHT11** temperature and humidity sensor (GPIO4),
- **HC-SR04** ultrasonic distance sensor: Trig (GPIO23) and Echo (GPIO24),
- **Keypad buttons** for entering PIN: GPIO5, GPIO6, GPIO13, GPIO19, GPIO26,
- **Actuators:**
    - **SG90 servo motor** used as a physical door lock (GPIO18),
    - **LED indicators**: red (GPIO21) and green (GPIO20) for door status,
    - **Blue LED** representing light (GPIO16)

The Raspberry PI collects data from the DHT11, HC-SR04 and keypad buttons, and sends it to the Main Server. It works both as a client and as a server, enabling bidirectional communication for exchanging data such as temperature, humidity, motion status, brightness and lock status.

User

A person connected to the Main Server who can view real-time and historical data collected from sensors, as well as control output devices through a user-friendly web interface.

Main Database

runs on a MySQL, connected to the Main Server. It consists of tables:

- **Users** - stores usernames (for both Users and Devices), encrypted passwords using BCrypt, assigned roles, linked account, and the door code. Each User is paired with one Device in a 1:1 relationship.
- **Measurements** - stores time-stamped sensor data, including temperature, humidity, motion status, and the Device username.
- **States** - stores the current state of each Device, including light status and lock status.

The database provides persistent storage for both historical measurements and current device states, ensuring reliable communication with the Main Server.

Main Server

Runs on a Spring Boot framework in Java and is connected locally to the MySQL database. It supports concurrent connections from multiple Devices and Users. The server handles authentication, authorization, data collection, and bidirectional communication between Users and Devices.

It uses several Spring modules:

- **Spring Security** - handles login, session identifiers, authorization, and password encryption. Access to each endpoint depends on the role assigned to a User/Device.
- **Spring Data JPA** - ORM used to map database records into Java entity objects and perform operations on them.
- **Spring WebFlux** - used to build an internal HTTP client for sending data from the server to Devices.

The Main Server exposes multiple GET and POST endpoints and returns both HTML and JSON responses.

The main user interface is main.html, which uses JavaScript to display real-time temperature, humidity, and motion data. Through this interface, Users can also change the door lock and light state. The page additionally shows historical measurements and provides buttons for device testing and user logout.

2. Scheme*

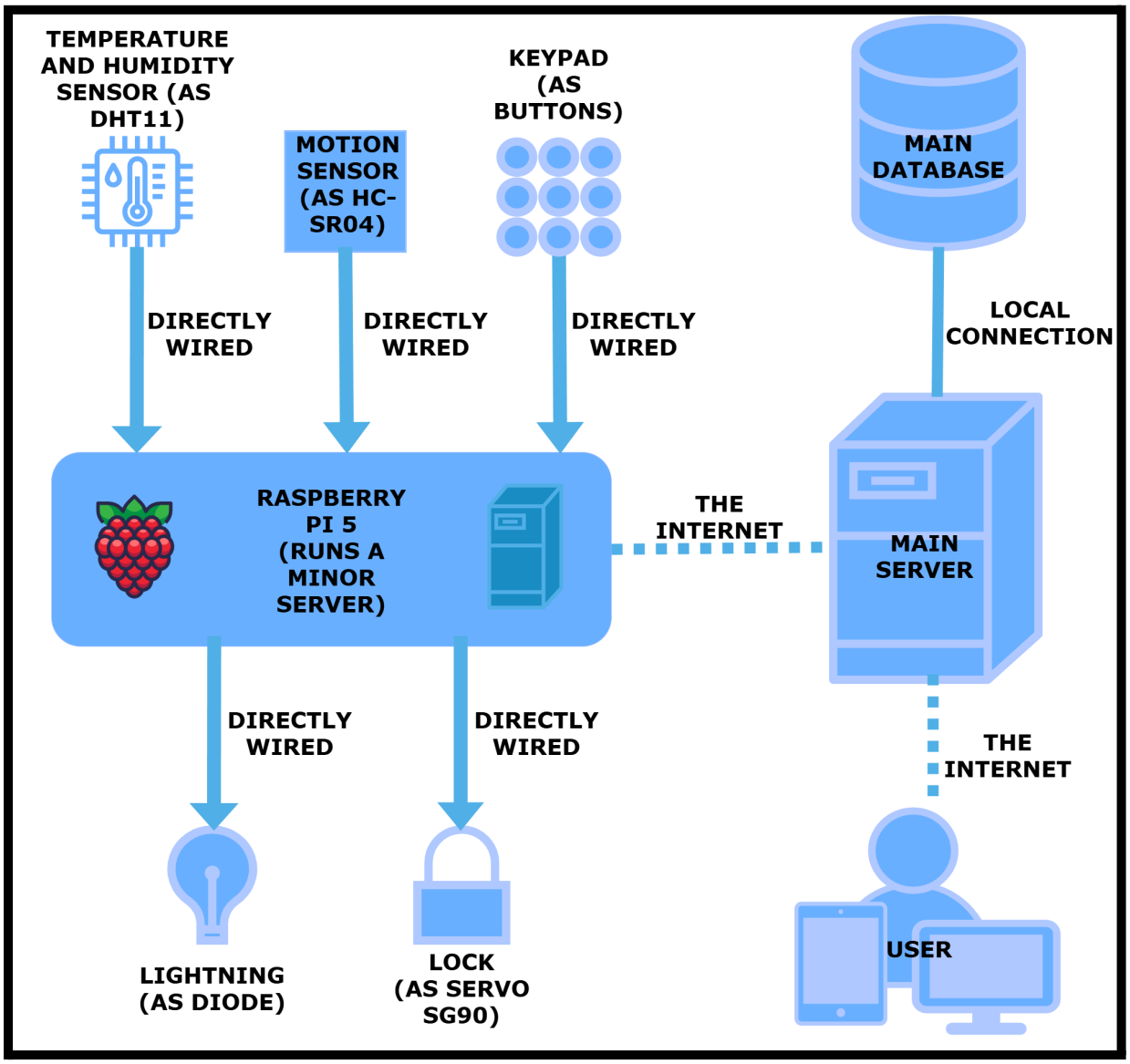


Figure 2. System architecture diagram illustrating communication between components.

** The main server provides a possibility of concurrent connection for multiple users and multiple circuits of Raspberry PI 5 with components.*

3. Design assumptions vs. implementation.

We aimed to create a smart-home system capable of seamless communication between multiple devices and their users. All planned requirements were successfully implemented, including sensor monitoring, remote device control, and secure door access. In addition, we extended the original design with an extra feature - adjustable lightning control. Our project might be developed further. It might contain more input and output devices such as a CCTV camera, or a basic alarm module. We could also add features like automatic lighting

schedules, notifications sent to the user using e-mail, history of changes, division into areas with different input and output devices, monitoring data or simple energy-usage monitoring. We could also let a user choose which elements he wants to control and read data from on his main webpage.

4. Code listing.

4.1. Main Server (Spring Boot)

```java
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{
        http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        // URL /device/** available only for role "device".
                        .requestMatchers("/device/**").hasRole("device")
                        // URL /user/** available only for role "user".
                        .requestMatchers("/user/**").hasRole("user")
                        // Any request to any URL needs authenticated user.
                        .anyRequest().authenticated()
                )
                // Login form on /login URL available for every user.
                .formLogin(form -> form
                        .permitAll()
                )
                // Every user is able to logout.
                .logout(logout -> logout.permitAll()
                );

        return http.build();
    }
    // Method for password encoding
    @Bean
    public PasswordEncoder passwordEncoder(){
        return new BCryptPasswordEncoder();
    }
}
```

Figure 3. Configuration for Spring Security to provide authorization and authentication.

```java
// Mapping URL to a method of an API controller.
    @GetMapping("/user/main/state")
    // Method getStatus with argument "principal" from dependency injection.
    public State getStatus(Principal principal){
        // Using "userRepository" from DI to make a query to retrieve record from "Users" table
        Optional<User> user = userRepository.findByUsername(principal.getName());
        if (user.isPresent()){
            // Getting "connectedUsername" (device username) that is assigned to a specific user.
            String connectedUsername = user.get().getConnectedUsername();
            // Finding the state assigned to the device username;
            Optional<State> state = stateRepository.findByusername(connectedUsername);
            if (state.isPresent()){
                // Returning JSON file of a state for a specific device.
                return state.get();
            }
        }
        return new State();
    }
```

Figure 4. Mapping for API controller to retrieve the newest state of the Raspberry Pi input devices.

```java
@Entity
@Table(name = "users")
public class User {
    // Attributes of the "users" table.
    @Id
    private String username;
    private String password;
    private String role;
    @Column(name = "connected_username")
    private String connectedUsername;

    // Getters and setters
    // ...
}
```

Figure 5. Entity annotation in Spring Boot to map database records of “users” table into Java objects (and vice versa).

4.2. Code for Raspberry PI 5

```python
# Login to the web server and initialize device settings
def login():
    # Authenticate and start session
    resp = config.SESSION.post(
        config.SPRING_URL_LOGIN,
        data={"username": config.VALID_USERNAME, "password": config.VALID_PASSWORD},
        headers={"Content-Type": "application/x-www-form-urlencoded"}
    )

    config.JSESSIONID = config.SESSION.cookies.get("JSESSIONID")

    # Fetch initial device config from backend
    init = config.SESSION.post("http://192.168.1.12:8080/device/init").json()

    # Extract initial parameters
    brightness = init.get("brightness", 0)
    lockStatus = init.get("lockStatus", False)
    code = init.get("doorCode", "")
    # ...apply settings...
```

Figure 6. Login function that authenticates the Raspberry Pi with the web server, retrieves device configuration, and initializes local settings.

```python
def jsonWithSessionCookie(data):
    resp = make_response(jsonify(data))
    if config.JSESSIONID:
        resp.set_cookie("JSESSIONID", config.JSESSIONID, path="/", httponly=True)
    return resp
```

Figure 7. Helper function that returns a JSON response and attaches the current session cookie so the Raspberry Pi stays authenticated when communicating with the server.

```python
# Handle door code change and send status to the web application
@code_bp.route("/user/code", methods=["POST"])
def update_code():
    # ...algorithm...

# Handle opening/closing the door remotely and send status to the web application
@door_open_bp.route("/user/lock", methods=["POST"])
def openDoorRemote():
    # ...algorithm...

# Handle brightness change and send status to the web application
@light_bp.route("/user/light", methods=["POST"])
def setLight():
    # ...algorithm...

# Handle device test and send results to the web application
@test_bp.route("/user/test", methods=["POST"])
def deviceTest():
    # ...algorithm...
```

Figure 8. Python Flask endpoint handlers responsible for updating door codes, controlling the door and light and running device tests, each synchronizing the Raspberry Pi state with the web application.

```python
def sender():
    while True:
        # temp. and humidity readings
        temperature = getSensorValues(lambda: dht.temperature)
        humidity = getSensorValues(lambda: dht.humidity)

        # POST structure
        data = {
            "measurementTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "temperature": None if temperature is None else round(temperature, 2),
            "humidity": None if humidity is None else round(humidity, 2),
            "doorOpened": config.DOOR_OPENED,
            "motion": config.MOTION_DETECTED,
            "distance": config.MOTION_DISTANCE if config.MOTION_DETECTED else None
        }

        # POST to the web application with correct session
        try:
            resp = config.SESSION.post(
                config.SPRING_URL_DATA,
                json=data,
                timeout=3
            )
        except Exception as e:
            print(e)

        config.MOTION_DETECTED = False
        config.MOTION_DISTANCE = None

        sleep(10)
```

Figure 9. Python routine that reads sensor data, prepares a JSON payload, and periodically sends it to the web server.

```python
# Handle sending door status to the web application when physically opened/closed
def doorSendInfo():
    try:
        # Create data object for the POST request
        data = {
            "measurementTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "doorOpened": not(config.DOOR_OPENED)
        }

        # Send data to the web application with proper session cookie
        config.SESSION.post(config.SPRING_URL_DOOR_STATUS, json=data, timeout=3)

    # Handle errors
    except Exception as e:
        print("Send error:", e)
```

Figure 10. Function responsible for sending updated door status from the Raspberry Pi to the web application using an authenticated session.

5. Photos of the completed layout.

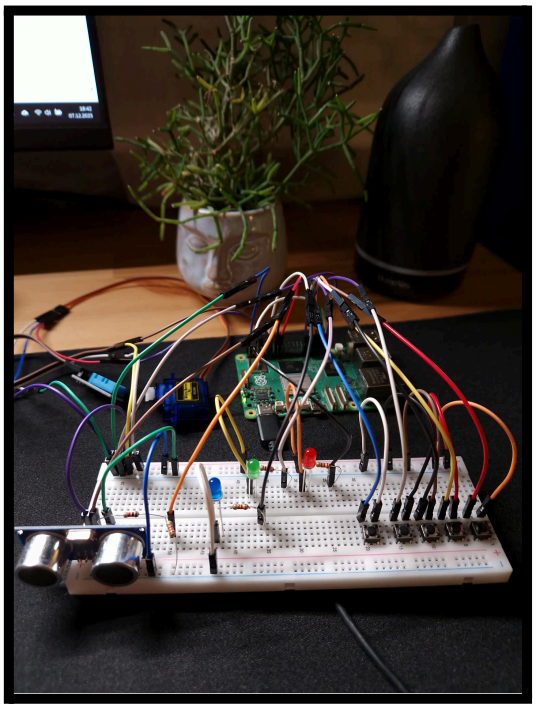


Figure 11.

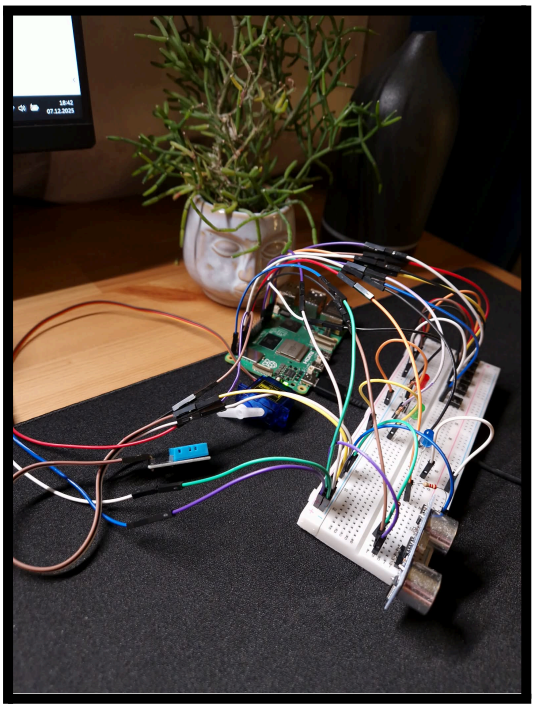


Figure 12.

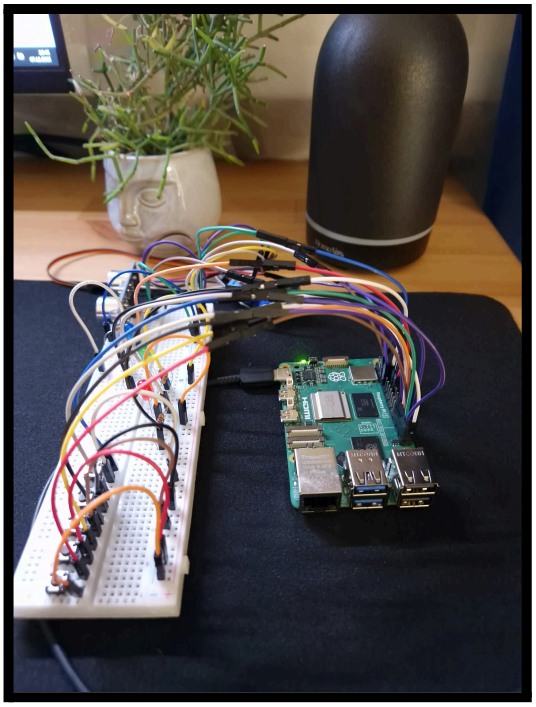


Figure 13.

6. Screenshots of the application.

Figure 14. Form to sign in on a website.

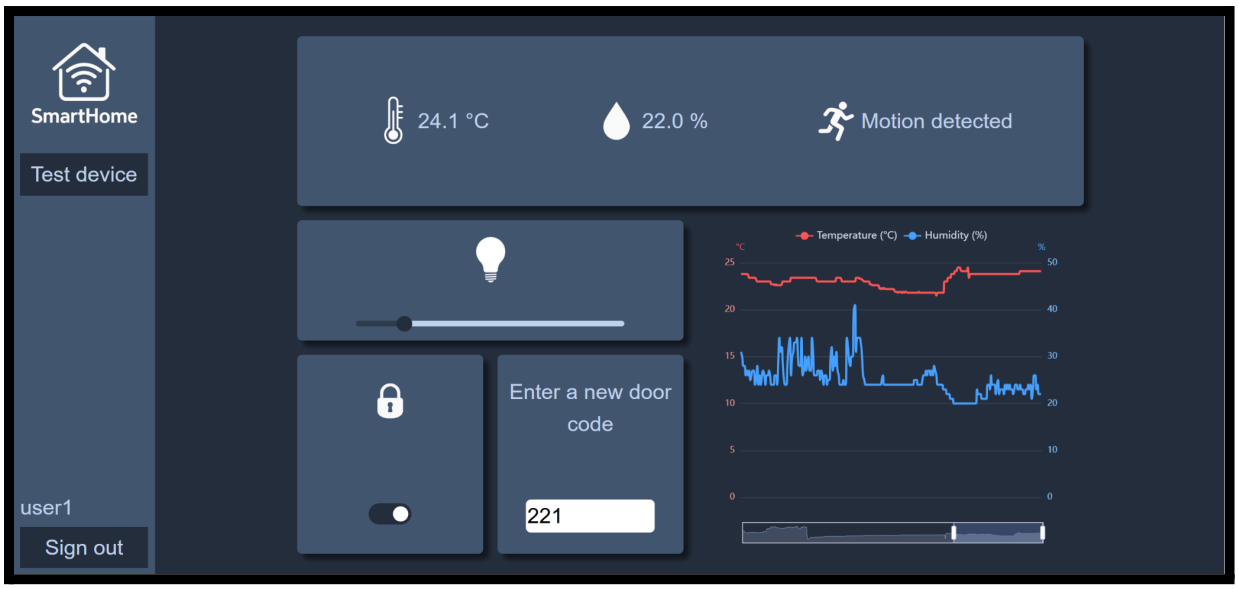


Figure 15. Main webpage for a user.

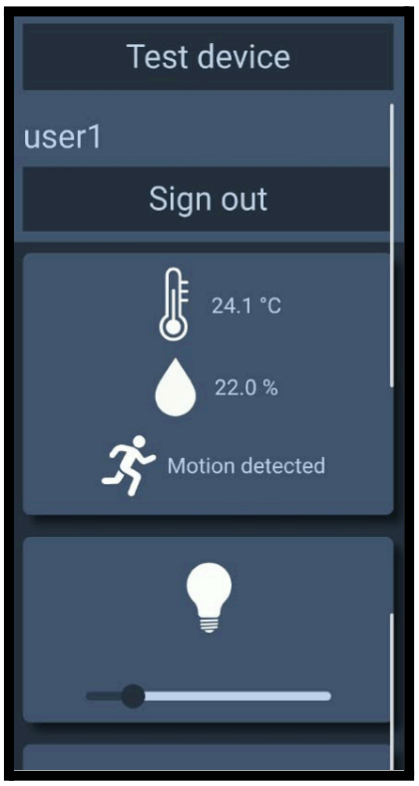


Figure 16. Part of a main webpage for a user for a mobile device.

7. Summary and conclusions.

We designed and implemented a complete smart-home platform that integrates hardware sensing, local device control, and remote management via a web application. By combining Raspberry Pi 5 with environmental sensors, a custom keypad, LEDs, and a servo-driven door mechanism, we achieved a functional and reliable system capable of collecting real-time data and executing user-initiated actions. Communication with the Spring Boot server enables continuous synchronization of states such as temperature, humidity, motion detection, lock status, and lighting. All collected measurements are stored in a MySQL database, making them accessible for later analysis through the web interface.

Throughout development, we encountered various challenges — sensor noise, request synchronization, ensuring consistent door-lock behavior, and dealing with hardware concurrency issues. These were resolved using debouncing, mutex locks, session cookies, improved error handling, and separation of background worker threads. We also implemented techniques such as averaging trimmed sensor samples to increase measurement stability. All of these improvements resulted in a system that is both robust and user-friendly, while maintaining reliable operation even under inconsistent hardware conditions.

In conclusion, the project met all initial objectives and even expanded beyond the core requirements by adding extra features like light brightness control, device test routines, and enhanced real-time synchronization. The work provided valuable hands-on experience with IoT system design, backend-frontend communication, secure authentication, and hardware-software integration.